Agentic Systems Studio

Start Assignment

- Due Apr 24 by 11:59pm
- Points 100
- Submitting a text entry box

Agentic Systems Studio

Two tracks. Three phases. One portfolio-ready project.

What you are building

Your team will design and build a portfolio-ready agentic system or agentic experience that applies the concepts of the course to a real problem.

The project is completed in three phases and should show clear problem framing, justified architecture, realistic evaluation, honest failure analysis, and professional documentation.

This team project is worth 45% of the course grade and is split across three phases: Phase 1 (5%), Phase 2 (15%), and Phase 3 (25%). Final presentations will be scheduled during the university-designated exam period. Submit all written deliverables by 11:59 PM on the due date unless Canvas states otherwise.

Guiding principle

You are not graded for using the most expensive tools or building the largest system.

You are graded for making strong design choices and supporting them with evidence.

Overview

This project is the capstone of the course. It is designed to help you translate an idea into an auditable artifact that you could show in a portfolio, discuss in an interview, or reuse in a professional setting.

Strong projects frame a real problem clearly, justify why an agentic approach is appropriate, choose an architecture carefully, test the system honestly, and communicate the result in a professional way.

Across all phases, focus on the same core discipline emphasized in the course: define success criteria up front, build and test in realistic scenarios, use traces and evaluations to diagnose failures, and produce the professional artifacts that matter in practice, including architecture diagrams, evaluation plans, governance controls, and evidence logs.

Deliverables at a glance

Phase	Weight	Purpose	Core deliverables
Phase 1 Scoping and Justification	5%	Define the problem, user, scope, and architectural direction.	Project summary; target user; problem statement; agentic justification; initial architecture; success criteria; scope boundaries; risks; team plan.
Phase 2 Architecture, Prototype, and Evaluation Plan	15%	Move from idea to a testable design and partial implementation or prototype.	Architecture diagram; role definitions; coordination logic; tools/memory/data design; prototype; evaluation plan; risk and governance plan; contribution update.
Phase 3 Final Product, Evidence, and Reflection	25%	Deliver the final artifact with evidence, failure analysis, and polished documentation.	Final artifact; 5-minute video; final report; evidence package; failure analysis; individual reflection; portfolio-ready project package.

Project tracks

Choose one track. Both tracks are rigorous. The difference is the implementation path, not the standard of thinking, evaluation, or documentation.

Area	Track A: Technical Build	Track B: Applied Agent Experience
What you build	A functioning agentic system with meaningful coordination among roles, agents, or components. This usually includes tools, memory or state, orchestration logic, evaluation, and evidence logs.	An interactive agentic application, simulation, decision-support experience, workflow, or narrative system built with low-code, no-code, lightweight coding, or mixed tools. It must still show clear agentic roles, coordination logic, evaluation, and governance thinking.
Best fit	Teams that want a more engineering-heavy portfolio piece and are comfortable building, testing, and debugging technical systems.	Teams that want to focus on product design, user experience, interaction logic, workflow, or simulation design without making traditional software engineering the center of the project.

What every project must include

- A real problem, user, stakeholder, or decision context
- A clear explanation of why an agentic approach is appropriate
- A justified architecture with distinct roles, components, branches, or handoffs
- A description of tools, data, memory, or state where relevant
- An evaluation plan with clear success criteria
- A discussion of risks, limitations, and governance
- Evidence of testing, failure analysis, and iteration
- A final artifact that is polished enough to show in a portfolio

How projects will be judged

Complexity must earn its place. Do not add agents, branches, or modules simply because they sound advanced.

A polished demo is not enough. You need evidence from realistic scenarios, including at least two failure cases.

Phase 1: Scoping and Justification

Weight: 5%

Goal

Define the problem, identify the user or stakeholder, explain the value of the project, and justify why an agentic approach makes sense.

Required deliverables

- Project title
- Team members
- Selected track: Track A or Track B
- One-paragraph project summary
- One page “Multi-agent System Canvas”
- One page “Agent Canvas” for each agent or component
- Target user, stakeholder, or audience
- Problem statement
- Why this is an agentic problem
- Initial architecture concept
- Initial success criteria
- Scope boundaries: what is in and what is out
- Initial risks or concerns
- Short team plan: who will lead which part

Track-specific additions

- **Track A:** Include your preliminary agent or component roles, likely tools or APIs, and a brief explanation of why the design should involve multiple interacting components.
- **Track B:** Include an initial user journey or interaction map, the roles or actors in the experience, and a brief explanation of where “agency” lives in the design.

What strong work looks like

- Specific problem, not a vague domain
- Clear user and use case
- Convincing explanation of why agency matters
- Realistic boundaries
- Concrete evaluation criteria, even if preliminary

Recommended effort: About 4 to 6 hours per student.

Phase 2: Architecture, Prototype, and Evaluation Plan

Weight: 15%

Goal

Move from idea to a concrete, testable design and a partial implementation or prototype.

Required deliverables

- Architecture diagram
- Role definitions: what each role, agent, or component does; what inputs it receives; what outputs it produces
- Coordination logic: who starts, how handoffs happen, how the system decides what to do next, where a human may intervene, and how the process stops
- Tools, memory, and data design
- Prototype or partial build
- Evaluation plan with at least 5 planned test scenarios or cases, success criteria, and measures
- Risk and governance plan
- Short contribution update

Track-specific additions

- **Track A:** Include a working prototype of the core flow, at least one interaction trace, log, or walkthrough, and a brief explanation of why this architecture is better than a simpler alternative.
- **Track B:** Include a working prototype of the interaction flow, branch logic or workflow logic, and a brief explanation of how the design operationalizes course concepts rather than merely presenting content.

What strong work looks like

- Architecture is easy to follow
- Extra complexity is justified
- The prototype already reveals the intended workflow
- The evaluation plan includes realistic and difficult cases
- Risks are concrete, not generic

Recommended effort: About 10 to 14 hours per student.

Phase 3: Final Product, Evidence, and Reflection

Weight: 25%

Goal

Deliver the final artifact and show evidence that it works, where it fails, and how you improved it.

Required deliverables

- Final artifact
- Final presentation or demo
- One 5-minute project video
- Final report
- Evidence package
- Failure analysis
- Reflection on improvements and next steps
- Individual contribution reflection

Evidence package

- At least 5 completed test scenarios or evaluation cases
- Results for each case
- At least 2 failure cases
- What changed after testing
- Supporting artifacts such as traces, screenshots, logs, outputs, diagrams, or user observations

Track-specific additions

- **Track A:** Include a runnable system, repository, or live demo; traces, logs, or outputs showing the interaction among agents or components; and evaluation evidence such as task success, quality, cost, latency, or stability.
- **Track B:** Include a usable interactive artifact, scenario-based or user-based evaluation, and evidence showing where users or test scenarios revealed ambiguity, breakdown, or needed redesign.

Final report should include

- Problem and user
- Architecture and design choices
- Implementation or build summary
- Evaluation setup
- Results
- Failure analysis
- Governance and safety reflection
- Lessons learned and future improvements

Recommended effort: 18 to 28 hours per student;

Grading rubric

Your project will be graded on the quality of your problem framing, the strength of your architecture and implementation choices, the rigor of your evaluation, and the professionalism of your final submission. Each phase will be scored on a 100-point rubric, then scaled to its course weight (Phase 1 = 5% of the final grade, Phase 2 = 15%, Phase 3 = 25%). You are not graded for choosing the most technically complex approach. You are graded for making good decisions and supporting them with evidence.

Phase 1 rubric

Category	Points (out of 100)	**What strong work shows**
Problem framing and user clarity	20	The project identifies a real problem, a clear user or stakeholder, and a believable reason the problem matters.
Agentic justification	20	The submission explains why an agentic approach is appropriate and why the proposed architecture makes sense for the task.
Scope discipline and feasibility	20	The project is ambitious but realistic, with a clear statement of what is in scope and what is out of scope.
Initial success criteria and risks	20	The submission identifies meaningful success criteria and early risks, limitations, or concerns.
Quality of submission and planning	20	The brief is well organized, complete, and specific enough to guide the next phase.

Total: 100 points (scaled to 5% of final grade)

Phase 2 rubric

Category	Points (out of 100)	What strong work shows
Architecture and role design	25	The project has a coherent architecture. Roles, agents, or components are distinct and purposeful. Handoffs, coordination, and stopping conditions are clear.
Tools, memory, data, and state design	20	The submission explains what information or tools each role can access, how memory or state is handled, and why those decisions make sense.
Prototype quality	15	The prototype demonstrates the core workflow and is concrete enough to test the main idea.
Evaluation plan and test design	20	The team proposes meaningful test scenarios, success criteria, and measures that go beyond a happy-path demo.
Risk and governance thinking	10	The submission identifies likely failure modes, misuse risks, or trust concerns and proposes realistic mitigations.
Documentation and contribution update	10	The materials are clear, complete, and professionally organized. Team roles and contributions are current.

Total: 100 points (scaled to 15% of final grade)

Phase 3 rubric

Category	Points (out of 100)	**What strong work shows**
Final artifact quality and completeness	20	The project works as intended at the level promised in earlier phases. The core workflow is functional, understandable, and reasonably polished.
Evidence of agentic behavior or coordination	15	The project shows meaningful interaction among roles, agents, components, branches, or decision structures. This is visible in the artifact, logs, traces, or interaction flow.
Evaluation quality and results	15	The team runs a meaningful evaluation set, reports results clearly, and interprets them honestly.
Failure analysis and iteration	15	The submission includes at least two concrete failure cases, explains why they happened, and shows what changed after testing.
Governance, trust, and responsible behavior	10	The system handles ambiguity, evidence gaps, safety boundaries, or user trust concerns thoughtfully.
Presentation (5-minute project video)	10	The presentation clearly explains the problem, architecture, main workflow, evidence layer, limitations, and key contribution of the project through a well-structured 5-minute video.

Category	Points (out of 100)	What strong work shows
Documentation, reproducibility, and portfolio quality	15	The final package is well organized and easy to navigate. A reviewer can quickly find the architecture, run instructions, screenshots, evaluation files, outputs, and presentation materials.

Total: 100 points (scaled to 25% of final grade)

Performance standards

- **Excellent:** Complete, specific, well justified, well tested, and professionally presented.
- **Good:** Mostly complete and convincing, with only minor gaps in clarity, evidence, or polish.
- **Partial:** The main idea is present, but important pieces are weak, incomplete, or insufficiently tested.
- **Insufficient:** The submission is unclear, incomplete, poorly justified, or missing required evidence.

Submission requirements

Your submission must make it easy for a reviewer to understand what you built, how it works, how it was evaluated, and how to reproduce or inspect the project.

How to submit

- Submit on Canvas a single PDF submission packet that links to or references the rest of your materials.
- Also submit a GitHub repository link or a compressed project folder if a repository is not possible.
- Include one 5-minute project video.
- Include any live demo link if applicable.
- List the names of all team members clearly on the submission packet and README.

What the PDF submission packet must contain

- Project title
- Team members
- Selected track
- One-paragraph project summary
- Repository or project-file link
- Link to the 5-minute video
- Final report or an attached final report
- Architecture diagram
- Screenshot index

- Evaluation summary
- List of submitted files and folders

5-minute video requirements

- State the problem and target user
- Give a quick explanation of the architecture
- Show the main workflow from beginning to end
- Show evidence of coordination, branching, or agent interaction
- Show the evidence layer, citations, logs, or supporting artifacts where relevant
- Include one example of a failure, limitation, or boundary behavior
- Show the final output, artifact, or export
- Spend most of the video showing the actual project rather than slides

Required project package

- **README.md:** A top-level README that lets a reviewer understand the project in about five minutes.
- **Final report PDF:** Problem, architecture, implementation or build choices, evaluation methodology, results, failure analysis, governance, lessons learned, and next steps.
- **Architecture diagram:** At least one diagram showing roles, agents, components, branches, inputs and outputs, coordination logic, and human-in-the-loop points if applicable.
- **Screenshots:** Include 4 to 8 UI or workflow screenshots with captions and reference them in the report.
- **Evaluation package:** Every project must include test cases, evaluation results, a failure log, and version notes.
- **AI usage disclosure:** Include an AI_USAGE.md file describing what tools were used, what they were used for, what was changed manually, and what was independently verified.
- **Outputs:** Include exported artifacts, sample runs, or representative outputs generated by the project.
- **Video:** Include the 5-minute project video or a text file containing the shareable video link.

README minimum contents

- Project title and short description
- Team members
- Selected track
- Setup instructions
- How to run or access the project
- Required dependencies or platforms
- Folder guide
- Summary of evaluation materials
- Summary of outputs included
- Known limitations

Screenshot expectations

- Home or landing screen
- Main interaction screen
- Evidence, citation, or source view
- Saved thread, history, or state view if relevant
- Artifact generation or export screen
- Evaluation or results screen
- One image showing a failure case or boundary case
- One image showing settings, controls, or filters if relevant

If your project is not screen-based, substitute workflow screenshots, story maps, Twine views, orchestration boards, or equivalent visual evidence.

Evaluation package requirements

- test_cases.csv or test_cases.md
- evaluation_results.csv
- failure_log.md
- version_notes.md

Recommended directory structure

```
team-project/
  README.md
  AI_USAGE.md
  requirements.txt or environment.yml or platform_notes.md
  docs/
    final_report.pdf
    architecture_diagram.pdf
    project_summary.pdf
    screenshots/
      01_home.png
      02_main_flow.png
      03_evidence_view.png
      04_history_or_state.png
      05_export_or_artifact.png
```

```
    06_evaluation_view.png
    screenshot_index.md
src/ or app/
  ...project files...
data/   (if applicable)
  raw/
  processed/
  manifest.csv or manifest.json
eval/
  test_cases.csv
  evaluation_results.csv
  failure_log.md
  version_notes.md
outputs/
  demo_outputs/
  exported_artifacts/
  sample_runs/
media/
  demo_video_link.txt
phase_submissions/
  phase1/
  phase2/
  phase3/
```

Track-specific notes

- **Track A:** In addition to the shared structure, Track A should usually include source code, run instructions, configuration or environment files, and logs or traces showing system behavior.

- **Track B:** In addition to the shared structure, Track B should usually include exported project files where possible, a shareable build or hosted link if available, platform notes, and screenshots or visual maps of branching, interaction logic, or system flow.

Final submission checklist

- README with run and navigation instructions
- Final report PDF
- Architecture diagram
- 4 to 8 screenshots with captions
- Evaluation files
- Failure log
- AI usage log
- Final artifact or app
- Exported outputs or artifacts
- 5-minute video
- Working repository or project folder

Appendix: lightweight templates

1. Evaluation file template

Your evaluation file can be a CSV, spreadsheet, or Markdown table. At minimum, include these fields:

- case_id
- case_type
- input_or_scenario
- expected_behavior
- actual_behavior
- outcome
- evidence_or_citation
- notes

1. Failure log template

- failure_id
- date
- version_tested
- what_triggered_the_problem
- what_happened
- severity
- fix_attempted
- current_status

1. Screenshot index template

- screenshot_file
- what_it_shows
- why_it_matters
- where_it_is_discussed_in_the_report

1. AI usage log template

- Tool name and version
- What you used it for
- Exact prompt or task given to the tool
- What you changed manually afterward
- What you verified independently

Course policies that still apply

- Use of generative AI tools is allowed, but it must be appropriately acknowledged and cited, including the specific version of the tool used.
- Submitted work should include the exact prompt used to generate the content as well as the AI tool's full response in an appendix.
- You are responsible for the accuracy and appropriateness of any AI-generated material that appears in your submission.

Final reminder

The strongest projects in this course will not be the ones with the most features.

They will be the ones that frame a real problem well, choose an architecture carefully, test the system honestly, and communicate decisions clearly.